Funzioni in C

Classe 3TRED — Liceo Scientifico Scienze Applicate

Indice degli argomenti

4. Funzioni: prototipo e uso
5. Passaggio parametri: **per valore** vs **per riferimento**
6. **Funzioni ricorsive** (con lo stack e tanti esempi)
7. Esempi completi (variabili semplici, poi array)
8. Errori comuni e cose da **non** fare

4. Funzioni in C

Perché usare le funzioni

Le funzioni servono a **spezzare un programma** in blocchi più piccoli, riutilizzabili e più facili da capire e testare.

Una funzione ha bisogno di:

1. **Prototipo** (dichiarazione): dice al compilatore come si chiama la funzione, cosa restituisce, quali parametri accetta
2. **Definizione**: contiene il codice effettivo
3. **Chiamata**: il punto in cui la funzione viene eseguita

Prototipo di funzione

Il prototipo si scrive di solito **prima del** `main`, per informare il compilatore prima ancora di vedere la definizione completa.

```
tipo_ritorno nome_funzione(tipo_param1, tipo_param2, ...);
```

Esempio:

```
int somma(int a, int b);          // prototipo: restituisce int, prende due int
void stampaMessaggio(void);       // prototipo: non restituisce nulla, non prende parametri
```

Il prototipo termina con **punto e virgola** (non ha corpo).

Definizione e chiamata

```c
#include <stdio.h>

int somma(int a, int b);   // prototipo

int main(void) {
    int risultato = somma(3, 5);   // chiamata alla funzione
    printf("Risultato: %d\n", risultato);
    return 0;
}

// definizione (può stare anche dopo il main, grazie al prototipo)
int somma(int a, int b) {
    return a + b;
}
```

`a` e `b` sono i **parametri formali**; `3` e `5` sono gli **argomenti** passati nella chiamata.

5. Passaggio dei parametri

Per valore vs per riferimento

Passaggio per valore: il concetto

In C, **di default**, i parametri vengono passati **per valore**:

> Quando chiami una funzione, viene creata una **copia** del valore dell'argomento. La funzione lavora sulla copia, **non sull'originale**.

Conseguenza pratica: **qualsiasi modifica fatta dentro la funzione al parametro NON si riflette sulla variabile originale** nel chiamante.

Esempio completo: passaggio per VALORE (variabile semplice)

```c
#include <stdio.h>

void raddoppia(int n);   // prototipo

int main(void) {
    int x = 10;
    printf("Prima della chiamata:  x = %d\n", x);

    raddoppia(x);   // passo il VALORE di x (una copia)

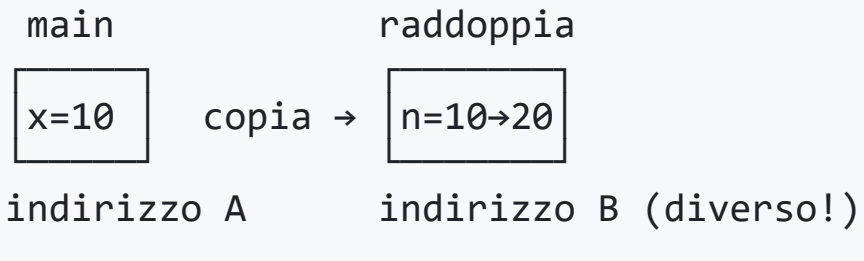
    printf("Dopo la chiamata:      x = %d\n", x);
    return 0;
}

void raddoppia(int n) {
    n = n * 2;                          // modifico solo la copia locale n
    printf("Dentro la funzione:    n = %d\n", n);
}
```

Output e spiegazione

```
Prima della chiamata:  x = 10
Dentro la funzione:    n = 20
Dopo la chiamata:      x = 10
```

- `n` è una **copia locale** di `x`, con un proprio indirizzo di memoria diverso da quello di `x`
- Modificare `n` modifica solo quella copia
- Quando la funzione termina, `n` viene distrutta e `x` **resta invariato**



Passaggio per riferimento: il concetto

Per far sì che una funzione possa **modificare davvero** la variabile del chiamante, bisogna passarle **l'indirizzo** della variabile, cioè un **puntatore**.

> Passaggio "per riferimento" in C = passare **l'indirizzo** di una variabile (con `&`) a un parametro che è un **puntatore** (`*`).

La funzione, dereferenziando il puntatore (`*p`), può leggere **e scrivere** direttamente sulla variabile originale.

Esempio completo: passaggio per RIFERIMENTO (variabile semplice)

```c
#include <stdio.h>

void raddoppia(int *n);   // prototipo: parametro puntatore

int main(void) {
    int x = 10;
    printf("Prima della chiamata:  x = %d\n", x);

    raddoppia(&x);   // passo l'INDIRIZZO di x

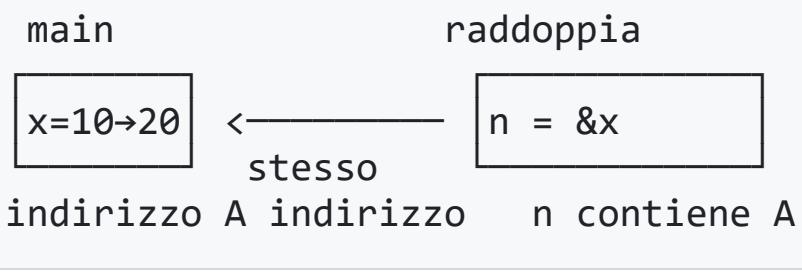
    printf("Dopo la chiamata:      x = %d\n", x);
    return 0;
}

void raddoppia(int *n) {
    *n = *n * 2;       // modifico il valore ALL'INDIRIZZO puntato da n -> modifico x!
    printf("Dentro la funzione:    *n = %d\n", *n);
}
```

Output e spiegazione

```
Prima della chiamata:  x = 10
Dentro la funzione:    *n = 20
Dopo la chiamata:      x = 20
```

- Questa volta `n` **non è una copia di x**: `n` contiene l'**indirizzo** di `x`
- `*n = *n * 2` significa: "vai all'indirizzo memorizzato in `n` (che è quello di `x`) e raddoppia il valore lì contenuto"
- Il risultato è visibile anche in `main`, perché abbiamo modificato **la stessa cella di memoria**



Confronto riassuntivo

	Per valore	Per riferimento
Cosa viene passato	una **copia** del valore	l'**indirizzo** (puntatore)
Parametro nel prototipo	`int n`	`int *n`
Chiamata	`funzione(x)`	`funzione(&x)`
Accesso nel corpo	`n`	`*n`
La funzione può modificare l'originale?	❌ No	✅ Sì
Memoria occupata	nuova cella indipendente	condivide la cella con l'originale

Passaggio di ARRAY alle funzioni

Un caso particolare importante

Quando passi un **array** a una funzione, in C **passi sempre, di fatto, un puntatore** al primo elemento (perché il nome dell'array *è* un puntatore, come visto prima).

> 📌 Gli array vengono **sempre passati "per riferimento"**, anche se non si usa esplicitamente `&` o `*` nella chiamata!

Questo significa che una funzione **può sempre modificare gli elementi di un array ricevuto**, a differenza di quanto avviene con una singola variabile passata per valore.

Esempio completo: modificare un array in una funzione

```c
#include <stdio.h>

void raddoppiaArray(int arr[], int dimensione);  // prototipo

int main(void) {
    int numeri[5] = {1, 2, 3, 4, 5};

    printf("Prima: ");
    for (int i = 0; i < 5; i++) printf("%d ", numeri[i]);
    printf("\n");

    raddoppiaArray(numeri, 5);   // passo l'array (= indirizzo del 1° elemento)

    printf("Dopo:  ");
    for (int i = 0; i < 5; i++) printf("%d ", numeri[i]);
    printf("\n");

    return 0;
}

void raddoppiaArray(int arr[], int dimensione) {
    for (int i = 0; i < dimensione; i++) {
        arr[i] = arr[i] * 2;   // modifico direttamente gli elementi originali!
    }
}
```

Output

```
Prima: 1 2 3 4 5
Dopo:  2 4 6 8 10
```

- `arr` nella funzione punta **alla stessa zona di memoria** dell'array `numeri` in `main`
- Non c'è nessuna copia dell'array: `arr[i] = ...` scrive direttamente sui dati originali
- Ecco perché, quando si passa un array, si passa **sempre di fatto anche la sua dimensione** come parametro separato (l'array "dimentica" quanto è lungo una volta passato alla funzione!)

Equivalenza tra notazioni

Il prototipo:

```
void raddoppiaArray(int arr[], int dimensione);
```

è **equivalente** a scrivere:

```
void raddoppiaArray(int *arr, int dimensione);
```

Sono due modi diversi di dire la stessa cosa: **"arr è un puntatore a int"**.
Dentro la funzione puoi usare sia `arr[i]` sia `*(arr + i)`: sono identici.

Esempio completo finale: funzione che calcola la somma di un array

```c
#include <stdio.h>

int sommaArray(int arr[], int dimensione);  // prototipo, sola lettura -> non modifica

int main(void) {
    int voti[4] = {7, 8, 6, 9};

    int totale = sommaArray(voti, 4);
    printf("Somma dei voti: %d\n", totale);
    printf("Media: %.2f\n", (float)totale / 4);

    return 0;
}

int sommaArray(int arr[], int dimensione) {
    int somma = 0;
    for (int i = 0; i < dimensione; i++) {
        somma += arr[i];
    }
    return somma;   // qui NON modifico l'array, restituisco solo un valore
}
```

💡 Anche se l'array è passato "per riferimento", una funzione può scegliere di **non modificarlo** e

Buona pratica: `const` per array non modificabili

Se una funzione riceve un array ma **non deve modificarlo**, è buona pratica dichiararlo `const`:

```
int sommaArray(const int arr[], int dimensione);
```

Il compilatore darà **errore** se, per sbaglio, scrivi `arr[i] = ...` dentro la funzione.
Questo rende il codice più sicuro e più chiaro da leggere.

6. Funzioni ricorsive

Una funzione che chiama se stessa

Cos'è la ricorsione

Una funzione è **ricorsiva** quando, per risolvere un problema, **richiama se stessa** su una versione **più piccola** dello stesso problema.

> 🪆 **Matrioska**: per sapere cosa c'è nel mezzo, apri la bambola. Dentro c'è... un'altra matrioska, **uguale ma più piccola**. Continui ad aprire finché trovi quella piena, che non si apre più.

L'idea in tre passi:

1. Se il problema è **banalissimo**, rispondi subito
2. Altrimenti **riduci** il problema a uno più piccolo
3. Risolvi quello più piccolo **con la stessa funzione**

Ricorsione nella vita di tutti i giorni (1)

🧍 **La fila al cinema**

Vuoi sapere *quante persone ci sono davanti a te*. Non puoi contarle, ma puoi chiedere:

- Tu chiedi a quello davanti: "Quanti siete davanti a te?"
- Lui chiede a quello davanti a lui... e così via
- Il **primo della fila** risponde: "**Zero!**"
- Ognuno, tornando indietro, risponde: "**quello che mi hanno detto + 1**"

Nessuno sa contare tutta la fila: **ognuno fa un pezzettino** e si fida di chi sta davanti.

Ricorsione nella vita di tutti i giorni (2)

🪜 Salire una scala di 10 gradini

Salire 10 gradini = salire **1 gradino** + *salire i restanti 9*.

Quando i gradini sono 0, hai finito.

📂 Cercare un file sul PC

Cerca nella cartella = guarda i file + *cerca in ogni sottocartella* (stessa operazione!).

Si ferma quando non ci sono più sottocartelle.

📖 Cercare una parola nel dizionario

Apri a metà: la parola è prima o dopo? Scarti metà dizionario e **ripeti** sulla metà rimasta.

Le due regole d'oro

Ogni funzione ricorsiva **deve** avere:

1. **Caso base** 🛑: la situazione più semplice, in cui la risposta è immediata e **non si richiama la funzione**. È il "primo della fila", la matrioska piena.
2. **Passo ricorsivo** 🔁: la funzione richiama se stessa con un problema **più piccolo**, che si avvicina al caso base.

```
tipo funzione(parametri) {
    if (caso_base)
        return risultato_semplice;      // 🛑 si ferma qui

    return ... funzione(problema_piu_piccolo) ...;   // 🔁 si richiama
}
```

⚠️ Senza caso base la funzione non si ferma mai (vedi errori comuni).

Primo esempio: conto alla rovescia

```c
#include <stdio.h>

void contoAllaRovescia(int n);   // prototipo

int main(void) {
    contoAllaRovescia(3);
    return 0;
}

void contoAllaRovescia(int n) {
    if (n == 0) {                // 🛑 caso base
        printf("VIA!\n");
        return;
    }
    printf("%d\n", n);
    contoAllaRovescia(n - 1);    // 🔁 problema più piccolo
}
```

Output:

```
3
2
1
```

Prima o dopo la chiamata ricorsiva?

Cambia **dove** metti il `printf` e cambia l'ordine dell'output!

```
void conto(int n) {
    if (n == 0) return;
    printf("%d ", n);      // PRIMA della chiamata
    conto(n - 1);
}
// conto(3) stampa: 3 2 1
```

```
void conto(int n) {
    if (n == 0) return;
    conto(n - 1);
    printf("%d ", n);      // DOPO la chiamata
}
// conto(3) stampa: 1 2 3
```

Nel secondo caso il `printf` viene eseguito solo **al ritorno**, quando le chiamate si "srotolano". Tra poco vedremo perché, con lo **stack**.

Esempio classico: il fattoriale

In matematica: `n! = n · (n-1) · (n-2) · ... · 1`, con `0! = 1`.

Si può riscrivere in modo **ricorsivo**:

- `0! = 1` (caso base)
- `n! = n · (n-1)!` (passo ricorsivo)

```
int fattoriale(int n) {
    if (n == 0)                     // 🛑 caso base
        return 1;
    return n * fattoriale(n - 1);   // 🔁 passo ricorsivo
}
```

Il codice è **identico alla definizione matematica**: è questo il bello della ricorsione!

Fattoriale: cosa succede passo passo

```c
int main(void) {
    printf("%d\n", fattoriale(4));   // stampa 24
    return 0;
}
```

Andata (si espande, nessun calcolo è ancora possibile):

```
fattoriale(4) = 4 * fattoriale(3)
                3 * fattoriale(2)
                    2 * fattoriale(1)
                        1 * fattoriale(0)
                            1        <-- caso base!
```

Ritorno (si risolve dal basso verso l'alto):

```
fattoriale(1) = 1 * 1 = 1
fattoriale(2) = 2 * 1 = 2
fattoriale(3) = 3 * 2 = 6
fattoriale(4) = 4 * 6 = 24
```

🧱 Come funziona lo stack (1): le chiamate si impilano

Ogni volta che una funzione viene chiamata, il computer riserva un blocco di memoria chiamato **stack frame**, che contiene:

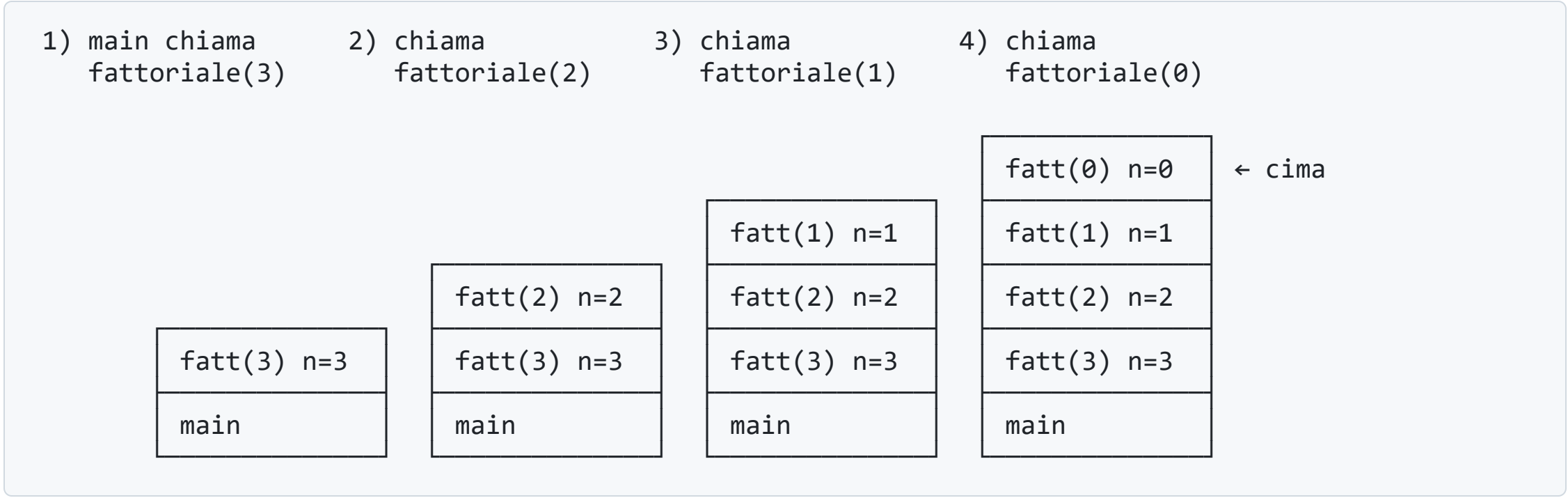
- i **parametri** ricevuti (`n`)
- le **variabili locali**
- il punto in cui **tornare** quando la funzione finisce

I frame si accumulano nello **stack** (la "pila"), come **piatti da lavare**: l'ultimo messo sopra è il primo che viene tolto (**LIFO**: *Last In, First Out*).

Ogni chiamata ricorsiva ha **il suo frame con la sua copia di** `n` : per questo le chiamate non si confondono tra loro.

🧱 Come funziona lo stack (2): `fattoriale(3)` in azione

Andata: ogni chiamata mette un nuovo frame in cima alla pila



Le funzioni sotto sono **in attesa**: hanno lasciato il calcolo a metà (`3 * ???`).

🧱 Come funziona lo stack (3): il ritorno

Raggiunto il **caso base**, i frame vengono tolti **uno alla volta dalla cima**, e ognuno consegna il risultato a quello sotto:

```
fatt(0) ritorna 1  →  frame eliminato
fatt(1) calcola 1 * 1 = 1, ritorna 1  →  frame eliminato
fatt(2) calcola 2 * 1 = 2, ritorna 2  →  frame eliminato
fatt(3) calcola 3 * 2 = 6, ritorna 6  →  frame eliminato
main riceve 6 ✅
```

Ecco perché nell'esempio del `printf` *dopo* la chiamata ricorsiva i numeri uscivano in ordine inverso: il `printf` aspetta il suo turno nel frame, e viene eseguito solo quando i frame sopra sono stati rimossi.

💡 Lo stack ha **dimensione limitata** (di solito pochi MB): se si impilano troppi frame, il programma si blocca con uno **stack overflow**.

Somma dei numeri da 1 a n

Idea: `somma(n) = n + somma(n-1)`, con `somma(0) = 0`.

```c
#include <stdio.h>

int somma(int n);

int main(void) {
    printf("Somma 1..5 = %d\n", somma(5));
    return 0;
}

int somma(int n) {
    if (n == 0) return 0;          // 🛑 caso base
    return n + somma(n - 1);       // 🔁 passo ricorsivo
}
```

```
somma(5) = 5 + somma(4) = 5 + (4 + somma(3)) = ... = 5+4+3+2+1+0 = 15
```

🪜 È la **scala**: "faccio 1 gradino e poi risolvo il resto".

Ricorsione sugli array (1): somma degli elementi

Idea: la somma di un array = **ultimo elemento + somma di tutti gli altri**.

Il "problema più piccolo" è lo **stesso array con un elemento in meno**.

```
int sommaRic(int arr[], int n) {      // n = quanti elementi considerare
    if (n == 0) return 0;             // 🛑 array vuoto: somma 0
    return arr[n - 1] + sommaRic(arr, n - 1);   // 🔁 ultimo + resto
}

int main(void) {
    int voti[4] = {7, 8, 6, 9};
    printf("Somma: %d\n", sommaRic(voti, 4));   // 30
    return 0;
}
```

```
sommaRic(voti,4) = 9 + sommaRic(voti,3) = 9 + 6 + sommaRic(voti,2) = ... = 30
```

Ricorsione sugli array (2): massimo e stampa inversa

```c
int massimo(int arr[], int n) {
    if (n == 1) return arr[0];                  // 🛑 un solo elemento: è il max
    int maxResto = massimo(arr, n - 1);         // 🔁 max di tutti gli altri
    return (arr[n - 1] > maxResto) ? arr[n - 1] : maxResto;
}

void stampaInversa(int arr[], int n) {
    if (n == 0) return;                         // 🛑
    printf("%d ", arr[n - 1]);                  // stampo l'ultimo
    stampaInversa(arr, n - 1);                  // 🔁 poi gli altri
}
```

```c
int v[5] = {3, 9, 2, 7, 5};
printf("Max = %d\n", massimo(v, 5));   // Max = 9
stampaInversa(v, 5);                   // 5 7 2 9 3
```

📇 *Come il presidente di classe*: "il più alto tra tutti" = il più alto tra **me** e **il più alto di quelli che restano**.

Il massimo comun divisore (Euclide)

Un algoritmo di oltre 2000 anni fa, che è **già ricorsivo**:

- `mcd(a, 0) = a` (caso base)
- `mcd(a, b) = mcd(b, a % b)` (passo ricorsivo)

```
int mcd(int a, int b) {
    if (b == 0) return a;          // 🛑
    return mcd(b, a % b);          // 🔁
}
```

```
mcd(48, 18) → mcd(18, 12) → mcd(12, 6) → mcd(6, 0) → 6
```

Quattro righe di codice, nessun ciclo, e funziona per qualunque coppia di numeri.

Fibonacci: due chiamate ricorsive

Successione: `0, 1, 1, 2, 3, 5, 8, 13, ...` (ogni numero è la somma dei due precedenti)

```
int fib(int n) {
    if (n == 0) return 0;                  // 🛑 due casi base
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);        // 🔁 DUE chiamate ricorsive
}
```

Le chiamate formano un **albero**:

```
              fib(4)
            /        \
        fib(3)        fib(2)
        /    \        /    \
    fib(2) fib(1)  fib(1) fib(0)
    /   \
fib(1) fib(0)
```

⚠️ `fib(2)` viene calcolato **due volte**: per numeri grandi (es. `fib(40)`) diventa **lentissimo**. La ricorsione elegante non è sempre la più efficiente!

La vera potenza: ricerca binaria

Come si cerca un nome sull'**elenco telefonico**? Non si legge pagina per pagina: si apre a metà e si **scarta metà** dell'elenco, poi si ripete.

```
// arr deve essere ORDINATO. Cerca x tra gli indici sx e dx
int cerca(int arr[], int sx, int dx, int x) {
    if (sx > dx) return -1;                          // 🛑 non trovato
    int m = (sx + dx) / 2;
    if (arr[m] == x) return m;                       // 🛑 trovato!
    if (x < arr[m])
        return cerca(arr, sx, m - 1, x);             // 🔁 metà sinistra
    return cerca(arr, m + 1, dx, x);                 // 🔁 metà destra
}
```

📈 Con **1 000 000** di elementi servono **al massimo 20 passi**. Il problema si dimezza ad ogni chiamata!

Torri di Hanoi: il problema "impossibile" in 5 righe

Hai 3 pali (A, B, C) e **n dischi** impilati su A, dal più grande (sotto) al più piccolo (sopra). Devi spostarli tutti su C, **un disco alla volta**, senza mai mettere un disco grande sopra uno più piccolo.

Ragionamento ricorsivo ("*mi fido che sappia spostare n-1 dischi*"):

1. Sposta i **n-1** dischi in cima da A a B (usando C d'appoggio)
2. Sposta il disco più grande da A a C
3. Sposta i **n-1** dischi da B a C (usando A d'appoggio)

```
void hanoi(int n, char da, char a, char aux) {
    if (n == 0) return;                                  // 🛑
    hanoi(n - 1, da, aux, a);                            // 🔁 1)
    printf("Sposta disco %d da %c a %c\n", n, da, a);    //    2)
    hanoi(n - 1, aux, a, da);                            // 🔁 3)
}
```

Hanoi con 3 dischi: `hanoi(3, 'A', 'C', 'B')`

```
Sposta disco 1 da A a C
Sposta disco 2 da A a B
Sposta disco 1 da C a B
Sposta disco 3 da A a C
Sposta disco 1 da B a A
Sposta disco 2 da B a C
Sposta disco 1 da A a C
```

- 7 mosse = $2^3 - 1$. Con **n dischi** servono $2^n - 1$ mosse
- Risolvere lo stesso problema **senza** ricorsione richiede un codice molto più complicato
- 💡 Questa è la forza della ricorsione: **scomporre un problema difficile in problemi uguali ma più piccoli**

Ricorsione vs iterazione (cicli)

Tutto ciò che si fa con la ricorsione si può fare anche con un ciclo (e viceversa), ma:

	Ricorsione	Iterazione (for/while)
Idea	il problema si **ripete in piccolo**	si **ripete un blocco** di istruzioni
Leggibilità	ottima per problemi "a pezzi" (alberi, cartelle, Hanoi)	ottima per ripetizioni semplici
Memoria	ogni chiamata usa un **frame** sullo stack	poca memoria
Velocità	un po' più lenta (costo delle chiamate)	di solito più veloce
Rischio	**stack overflow**	cicli infiniti

Regola pratica: se la soluzione ricorsiva è molto più chiara (Hanoi, ricerca binaria, strutture ad albero) usa la ricorsione; se è un semplice conteggio (somma 1..n) un ciclo va benissimo.

Errori comuni con la ricorsione

❌ 1. Dimenticare il caso base

```
int somma(int n) {
    return n + somma(n - 1);      // non si ferma mai!
}
```

Come due **specchi uno di fronte all'altro**: l'immagine si ripete all'infinito. Il programma riempie lo stack e va in **stack overflow** (*Segmentation fault*).

❌ 2. Non avvicinarsi al caso base

```
int somma(int n) {
    if (n == 0) return 0;
    return n + somma(n + 1);      // n cresce: non arriva mai a 0!
}
```

❌ **3. Valori troppo grandi**: `fattoriale(13)` con `int` va in **overflow** (il risultato non ci sta). Per numeri grandi usa `long long`.

Riepilogo sulla ricorsione

- Una funzione **ricorsiva** richiama **se stessa** su un problema **più piccolo**
- Servono sempre un **caso base** 🛑 e un **passo ricorsivo** 🔁 che si avvicina al caso base
- Ogni chiamata ha il suo **stack frame** con i suoi parametri; i frame si impilano all'andata e si svuotano al ritorno (**LIFO**)
- Il codice può risultare **cortissimo ed elegante**: fattoriale, MCD, ricerca binaria, Hanoi
- Attenzione a **stack overflow** e a ricorsioni **inefficienti** (Fibonacci)

> 🧠 Per scrivere una funzione ricorsiva chiediti sempre:
> **1) Qual è il caso più semplice? 2) Come riduco il problema? 3) Mi fido che la funzione risolva la versione più piccola?**